

Name/NetID: Nathan Kariuki / nkari011

Session Date: May 18, 2026 (1-2 hours)

Objective: Set up the Phase 2c repo and get the three dense atomic variants working.

Attempts made: First thing was getting the repo set up. Copied graph.h, graph.cpp, bsp.h, bsp.cpp from Phase 2b since none of that changes for 2c. The CsrGraph and BspAlgorithm interfaces stay the same so I didn't touch them. Wrote a skeleton main.cpp that just loads the graph and prints the vertex count to confirm everything compiled cleanly before touching anything else.

Then started on the actual dense atomic variants. The idea is the same structure as the Phase 2b parallel dense versions but with the LockPool ripped out and replaced by atomic CAS. For BFS it was pretty clean since each vertex only gets discovered once, dist[v] goes from -1 to some hop count and that's it. So one CAS from -1 is enough and whoever wins is the discoverer:

```
int expected = -1; if (dist[v].compare_exchange_strong(expected, candidate)) { in_next[v] = 1; updated[tid] = 1; }
```

SSSP and CC gave me more trouble because state[v] can get lowered more than once in the same round. Had to use a load-then-retry loop instead of a single shot:

```
long long cur_val = dist[v].load(); while (candidate < cur_val) { if (dist[v].compare_exchange_weak(cur_val, candidate)) { in_next[v] = 1; updated[tid] = 1; break; } }
```

First compile attempt on the atomic vector failed because I used assign() which doesn't work on atomics since they're not copyable. Switched to a plain for loop for initialization and that fixed it.

Solutions/Workarounds: Replaced assign() with a manual initialization loop for all atomic vectors. Also had to be careful to call load() and store() explicitly rather than treating atomics like regular ints.

Commits: setup: copy graph and bsp from phase 2b, add skeleton main; feat: add bfs push dense atomic; feat: add sssp push dense atomic; feat: add cc push dense atomic

Experiments/Analysis: Ran all three dense atomic variants on soc-LiveJournal1-weighted.txt with source=0 and 4 threads:

BFS dense atomic (4t): 0.795 sec SSSP dense atomic (4t): 6.022 sec CC dense atomic (4t): 2.851 sec

Verified all three against the Phase 2a answer files and everything passed. Already noticeably faster than the Phase 2b locked versions just from dropping the mutex overhead.

Learning outcome: The difference between `compare_exchange_strong` and `compare_exchange_weak` is actually important here. Strong is better for BFS since it's a one-shot attempt and spurious failure would mean missing the discovery. Weak is fine inside the SSSP/CC retry loop since the loop catches spurious failures anyway and weak can be faster on some architectures. Also atomics not being copyable is an easy thing to forget until it bites you at compile time.